

Санкт-Петербургский государственный университет

Направление "Большие данные и распределенная цифровая платформа"

**Лабораторная работа по дисциплине
Алгоритмы и структуры данных**

**"Решение задачи о коммивояжере с помощью метода
муравьиного алгоритма."**

Выполнил:

Зайнуллин Мансур Альбертович

Группа: 23.Б16-пу

Руководитель:

Дик Александр Геннадьевич

ассистент кафедры компьютерного
моделирования

и многопоточных систем

Санкт-Петербург

2025

Оглавление

1	Цель работы	3
2	Теоретические основы	4
3	Описание программы	5
3.1	Структура проекта	5
3.2	Спецификация программы	5
3.2.1	Модуль graph.py	6
3.2.2	Модуль ant_algorithm.py	8
3.2.3	Модуль colorGUI.py	10
3.2.4	Модуль main.py	11
4	Рекомендации пользователю	12
4.1	Установка программы	12
4.2	Использование программы	13
4.2.1	Создание графа	13
4.2.2	Настройка параметров	13
4.2.3	Запуск алгоритма	14
4.2.4	Просмотр результатов	14
4.2.5	Очистка графа	14
5	Тестирование алгоритма и его модификации	15
5.1	Методология тестирования	15
5.2	Тестирование на графах разных размеров	16
5.2.1	Граф с 4 вершинами и 8 рёбрами	16
5.2.2	Граф с 20 вершинами и 186 рёбрами	18
5.2.3	Граф с 100 вершинами и 3395 рёбрами	20
5.2.4	Граф с 500 вершинами и 100000 рёбрами	22
5.3	Анализ влияния гиперпараметров (граф с 100 вершинами и 3000 рёбрами)	24
5.3.1	Количество муравьёв	24
5.3.2	Количество итераций	25
5.3.3	Alpha (влияние феромона)	26

5.3.4	Beta (влияние эвристики)	27
5.3.5	Rho (коэффициент испарения)	28
5.4	Анализ результатов	29
5.5	Сравнение муравьиного алгоритма с алгоритмом имитации отжига и методом ближайшего соседа	30
5.5.1	Граф с 4 вершинами и 8 рёбрами	30
5.5.2	Граф с 20 вершинами и 186 рёбрами	31
5.5.3	Граф с 100 вершинами и 3395 рёбрами	31
5.5.4	Граф с 500 вершинами и 100000 рёбрами	32
6	Вывод	33
7	Полезные ссылки	34
8	Листинг программы	35
8.1	main.py	36
8.2	graph.py	38
8.3	ant _a lgorithm.py	40
8.4	colorGUI.py	46

1 Цель работы

Целью данной работы является исследование особенностей решения задачи о коммивояжере с помощью муравьиного алгоритма.

2 Теоретические основы

Задача коммивояжера (TSP) в графе $G = (V, A)$ с весами $w(a)$, $a \in A$, требует найти гамильтонов цикл C минимальной длины: $L(C) = \sum_{a \in C} w(a) \rightarrow \min$. Цикл проходит все вершины V ровно раз с возвратом в начальную точку.

Муравьиный алгоритм (ACO) имитирует поведение муравьев, использующих феромоны для поиска пути. Муравьи выбирают вершину с вероятностью $p_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in J} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta}$, где τ_{ij} — феромон, $\eta_{ij} = 1/w(i, j)$ — эвристика, α и β — веса факторов, J — непосещенные вершины. Феромоны обновляются: $\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \Delta\tau$, где ρ — испарение, $\Delta\tau$ — вклад муравьев.

Модификация "Шаблон" применяет словарь, задающий для каждой вершины следующую с минимальным весом ребра. Муравей следует шаблону, если вершина доступна, иначе использует вероятностный выбор, что ускоряет алгоритм.

3 Описание программы

3.1 Структура проекта

Программа состоит из следующих модулей, каждый из которых выполняет определенную функцию:

- `main.py`: Точка входа. Инициализирует граф и запускает приложение.
- `colorGUI.py`: Реализует графический интерфейс с визуализацией графа и настройкой параметров алгоритма.
- `graph.py`: Содержит классы `Graph`, `Node`, `Edge` для представления ориентированного графа.
- `ant_algorithm.py`: Реализует муравьиный алгоритм с модификацией "Шаблон".

Модули взаимодействуют следующим образом: `main.py` создает граф и передает его в `colorGUI.py`, который вызывает функции из `ant_algorithm.py` для поиска гамильтонова цикла.

3.2 Спецификация программы

Спецификация описывает ключевые компоненты программы на основе документации кода.

3.2.1 Модуль graph.py

Класс Node

Table 1: Класс Node

Метод/Атрибут	Описание	Входные/Выходные данные
value	Уникальный идентификатор вершины	— → int
neighbors	Список смежных вершин	— → List [Node]
outgoing_edges	Исходящие рёбра	— → List [Edge]
x, y	Координаты для визуализации	— → float или None
visited	Флаг посещения вершины	— → bool
add_neighbor(node)	Добавить соседа	node (Node) → Обновление списка neighbors
sorted_edges()	Возвращает отсортированный список исходящих рёбер по весу	— → List [Edge] (отсортированный)

Класс Edge

Table 2: Класс Edge

Метод/Атрибут	Описание	Входные/Выходные данные
from_node	Начальная вершина ребра	— → Node
to_node	Конечная вершина ребра	— → Node
weight	Вес ребра	— → int

Класс Graph

Table 3: Класс Graph

Метод/Атрибут	Описание	Входные/Выходные данные
directed	Флаг ориентированности графа	— → bool (True по умолчанию)
nodes	Список всех вершин графа	— → List [Node]
edges	Список всех рёбер графа	— → List [Edge]
add_node(value)	Добавить вершину в граф	value (int) → Объект Node
add_edge(from_node, to_node, weight)	Добавить ребро в граф	from_node (Node), to_node (Node), weight (int) → Создание Edge
display()	Выводит все вершины и рёбра	— → Вывод в консоль

3.2.2 Модуль ant_algorithm.py

Класс Ant

Table 4: Класс Ant

Метод/Атрибут	Описание	Входные/Выходные данные
current_node	Текущая вершина муравья	— → Node
path	Пройденный путь	— → List [Node]
visited	Множество посещённых вершин	— → Set [Node]
template	Словарь шаблона (вершина → следующая)	— → Dict [Node, Node] или None
<code>__init__(start_node, template)</code>	Инициализирует муравья с начальной вершиной и шаблоном	start_node (Node), template (Dict или None) → Инициализация
<code>move(graph, alpha, beta)</code>	Выполняет один шаг муравья	graph (Graph), alpha (float), beta (float) → bool

Функции

Table 5: Функции модуля ant_algorithm.py

Функция	Описание	Входные/Выходные данные
initialize_pheromones	Инициализирует феромоны на рёбрах	graph (Graph), initial_pheromone (float) → Установка edge.pheromone
update_pheromones	Обновляет феромоны с испарением и добавлением	graph (Graph), ants (List[Ant]), rho (float), Q (float) → Обновление edge.pheromone
calculate_path_length	Вычисляет длину пути	graph (Graph), path (List[Node]) → int
get_template	Создаёт статический шаблон на основе минимальных весов	graph (Graph) → Dict[Node, Node]
build_result_graph	Создаёт результирующий граф из пути	graph (Graph), path (List[Node]) → Graph
ant_algorithm(...)	Реализует муравьиный алгоритм с модификацией "Шаблон"	Множество параметров → Tuple[Graph, int, str, str]

3.2.3 Модуль colorGUI.py

Класс LabApp

Table 6: Класс LabApp

Метод	Описание	Входные/Выходные данные
<code>__init__(self, graph)</code>	Создание интерфейса с графом	graph (Graph) → Инициализация GUI
<code>create_widgets()</code>	Создание всех элементов интерфейса	— → Построение GUI
<code>on_canvas_click(event)</code>	Обработка кликов для создания вершин/рёбер	event (координаты x,y) → Добавление Node/Edge
<code>start_algorithm()</code>	Запуск муравьиного алгоритма	— → Обновление результатов
<code>update_edges_table()</code>	Обновление таблицы рёбер с феромонами	— → Синхронизация данных
<code>draw_graph()</code>	Отрисовка графа на холсте	canvas, graph → Визуализация

3.2.4 Модуль `main.py`

Table 7: Функции модуля `main.py`

Функция	Описание	Входные/Выходные данные
<code>create_graph(num_vertices, num_edges, canvas_width, canvas_height)</code>	Создаёт граф с заданным количеством вершин и рёбер	Параметры → Graph
<code>main()</code>	Запускает приложение	— → Запуск GUI

4 Рекомендации пользователю

Эта инструкция поможет вам установить и использовать программу. Она написана для Ubuntu, но её легко адаптировать и под другие операционные системы.

4.1 Установка программы

1. **Запустите терминал:** Нажмите `Ctrl + Alt + T` на клавиатуре — откроется окно для ввода команд.
2. **Обновите систему:** Напишите `sudo apt update` и нажмите `Enter`. Введите пароль, если попросят.
3. **Установите Python 3.11:** Введите `sudo apt install python3.11` и нажмите `Enter`.
4. **Установите Git:** Введите `sudo apt install git` и нажмите `Enter`.
5. **Перейдите в домашнюю папку:** Введите `cd ~` и нажмите `Enter`.
6. **Скачайте программу:** Напишите `git clone https://github.com/MansurYa/4-sem-labs-for-algorithms-and-data-s` и нажмите `Enter`.
7. **Войдите в папку программы:** Введите `cd 4-sem-labs-for-algorithms-and-data-structures/Lab3` и нажмите `Enter`.
8. **Установите Tkinter:** Напишите `sudo apt install python3-tk` и нажмите `Enter`.
9. **Запустите программу:** Введите `python3.11 main.py` и нажмите `Enter`.

4.2 Использование программы

После запуска вы увидите окно программы с тремя областями: слева — настройки, в центре — график, справа — таблица связей. Вот как с ней работать.

4.2.1 Создание графа

- **Добавление точек:** Щелкните левой кнопкой мыши в области "Исходный граф" (верхняя часть центра окна), чтобы поставить точку. Каждая точка — это город, который нужно посетить.
- **Добавление связей:** Щелкните на одну точку, затем на другую. Появится окошко — введите число (например, 10) и нажмите ОК. Это вес связи между точками (расстояние). Повторите для всех нужных связей.

4.2.2 Настройка параметров

- В левой части окна есть поля с числами:
 - "Количество муравьёв" (например, 100) — сколько муравьев ищут путь.
 - "Количество итераций" (например, 300) — сколько раз они будут искать.
 - "Alpha (феромон)" (например, 1.0) — важность следов муравьев.
 - "Beta (эвристика)" (например, 2.0) — важность расстояния.
 - "Rho (испарение)" (например, 0.1) — как быстро исчезают следы.
 - "Q (константа)" (например, 100.0) — сила новых следов.
 - "Начальный феромон" (например, 0.1) — начальные следы на связях.
- Измените числа, если хотите (щелкните в поле, удалите старое число, введите новое).

- Поставьте галочку "Использовать шаблон", если хотите ускорить поиск (муравьи будут чаще выбирать короткие пути).

4.2.3 Запуск алгоритма

- Нажмите кнопку "Запустить алгоритм" в левой части окна. Программа начнет искать лучший путь. Это может занять от нескольких секунд до минуты, в зависимости от размера графа.

4.2.4 Просмотр результатов

- После завершения:
 - В поле "Полученный путь" появится порядок точек (например, "0-1-2-0") — это маршрут.
 - В поле "Длина пути" — число (например, 112) — общая длина маршрута.
 - В области "Результат" (нижняя часть центра) — рисунок найденного пути (синие точки и стрелки).
 - В таблице справа — связи между точками. Те, что входят в маршрут, отмечены галочкой (✓).

4.2.5 Очистка графа

- Нажмите кнопку "Очистить граф" между областями "Исходный граф" и "Результат", чтобы удалить все точки и связи и начать заново.

5 Тестирование алгоритма и его модификации

В данной главе представлены результаты тестирования базового муравьиного алгоритма (АСО) и его модификации с использованием шаблона для решения задачи коммивояжера. Исследованы производительность на графах различного размера и влияние гиперпараметров на качество и скорость решения.

5.1 Методология тестирования

Тестирование проводилось на ориентированных взвешенных графах с числом вершин 4 (8 рёбер), 20 (186 рёбер), 100 (3395 рёбер) и 500 (100000 рёбер). Для каждого графа выполнено по 5 запусков базового алгоритма и модификации. Использовались параметры: количество муравьёв = 100, итераций = 300, $\alpha = 1.0$, $\beta = 2.0$, $\rho = 0.1$, $Q = 100.0$, начальный феромон = 0.1. Оценивались средняя длина пути и время выполнения (в секундах). Влияние гиперпараметров изучалось на графе с 100 вершинами и 3000 рёбрами.

5.2 Тестирование на графах разных размеров

5.2.1 Граф с 4 вершинами и 8 рёбрами

Базовый алгоритм

Table 8: Базовый алгоритм (4 вершины)

№ испытания	Длина пути	Время выполнения (с)
1	112	0.2020
2	112	0.2181
3	112	0.2184
4	112	0.2153
5	112	0.2082

Найденный путь (испытание 1): 2-1-3-0-2

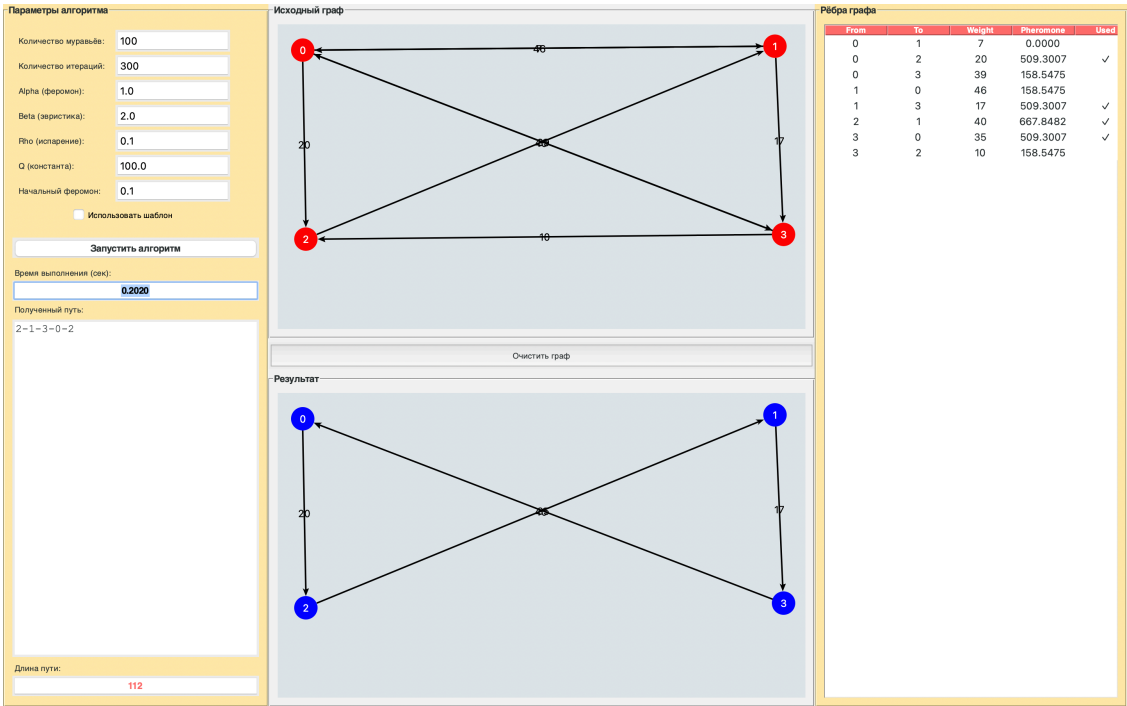


Рис 1. Граф с 4 вершинами (базовый)

Модификация (с шаблоном)

Table 9: Модификация с шаблоном (4 вершины)

№ испытания	Длина пути	Время выполнения (с)
1	112	0.1491
2	112	0.1478
3	112	0.1485
4	112	0.1478
5	112	0.1480

Найденный путь (испытание 1): 2-1-3-0-2

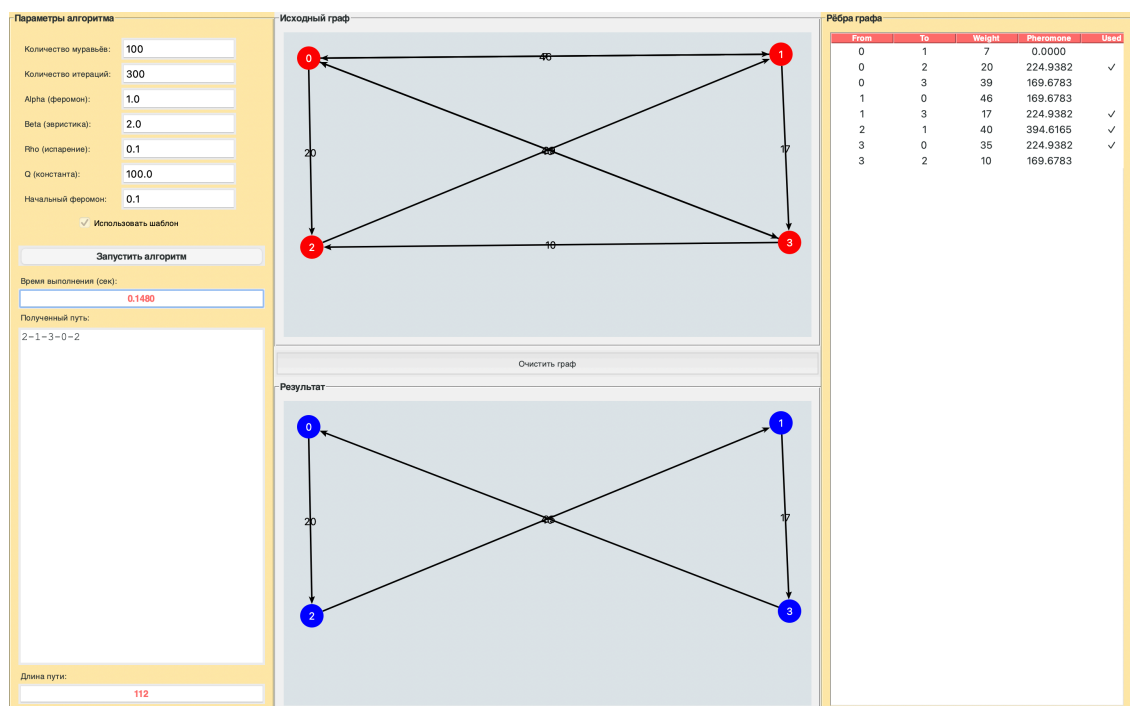


Рис 2. Граф с 4 вершинами (модификация)

5.2.2 Граф с 20 вершинами и 186 рёбрами

Базовый алгоритм

Table 10: Базовый алгоритм (20 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	134	1.2816
2	134	1.2777
3	142	1.2525
4	141	1.3142
5	134	1.4321

Найденный путь (испытание 1): 7-5-18-4-19-11-15-2-8-12-13-3-16-1-17-6-9-0-14-10-7

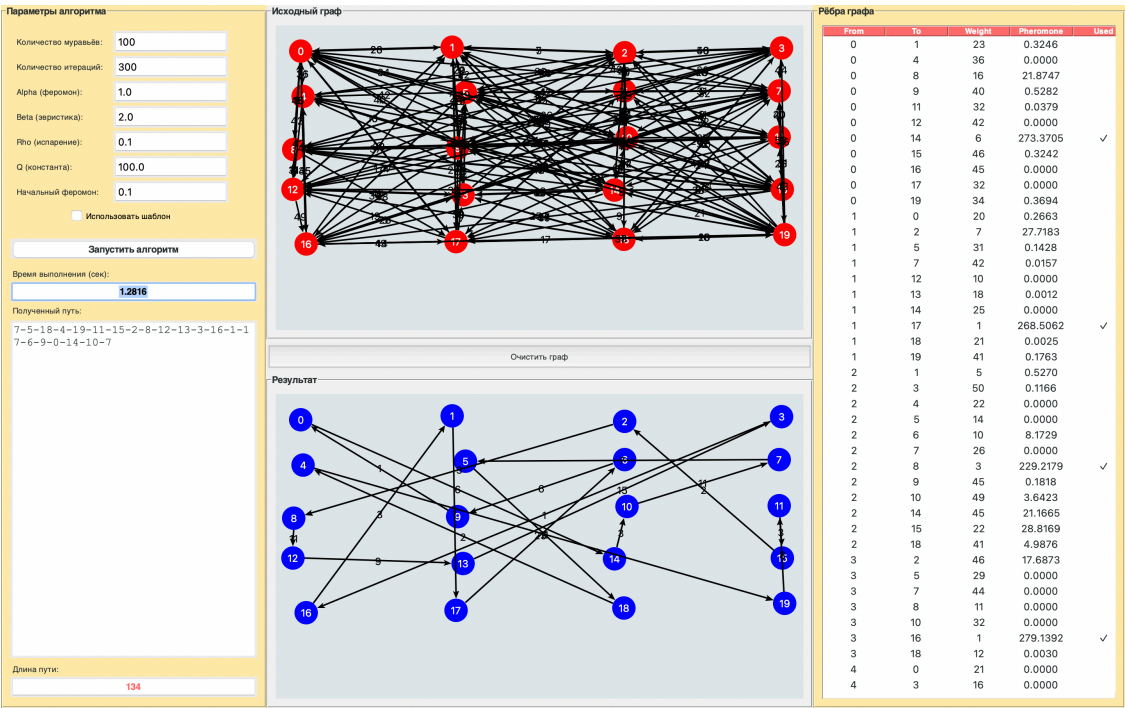


Рис 3. Граф с 20 вершинами (базовый)

Модификация (с шаблоном)

Table 11: Модификация с шаблоном (20 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	152	0.6689
2	152	0.6859
3	152	0.6433
4	152	0.6089
5	144	0.8473

Найденный путь (испытание 1): 5-18-4-10-12-7-13-3-16-1-17-6-9-0-14-15-2-8-19-11-5

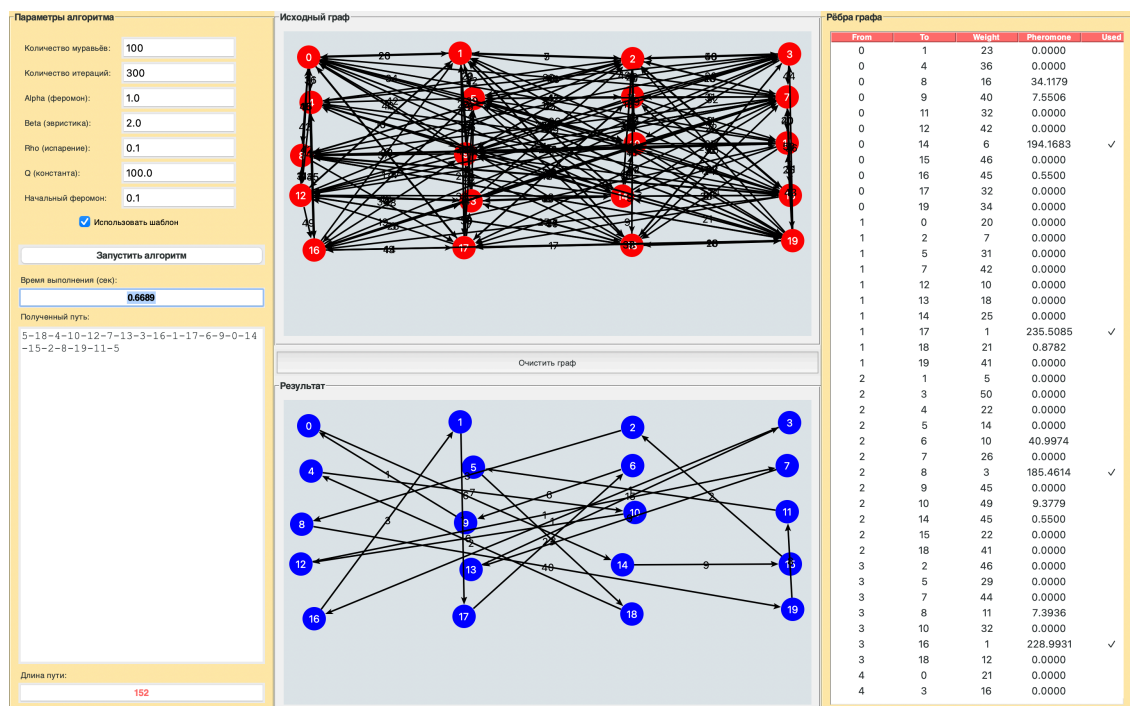


Рис 4. Граф с 20 вершинами (модификация)

5.2.3 Граф с 100 вершинами и 3395 рёбрами

Базовый алгоритм

Table 12: Базовый алгоритм (100 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	383	14.2781
2	377	14.3592
3	385	14.3056
4	380	14.3543
5	379	14.4311

Найденный путь (испытание 1): 0-25-39-74-16-19-68-57-43-17-98-82-36-92-76-24-99-70-58-28-91-54-71-33-38-65-64-31-11-73-69-21-90-8-67-83-56-87-30-15-51-34-4-41-53-94-18-37-52-32-85-89-2-6-62-44-66-49-55-59-96-10-12-47-84-93-61-48-72-3-35-95-80-63-7-78-5-22-40-14-1-29-27-42-45-46-60-77-26-9-79-50-81-97-13-88-75-23-86-20-0

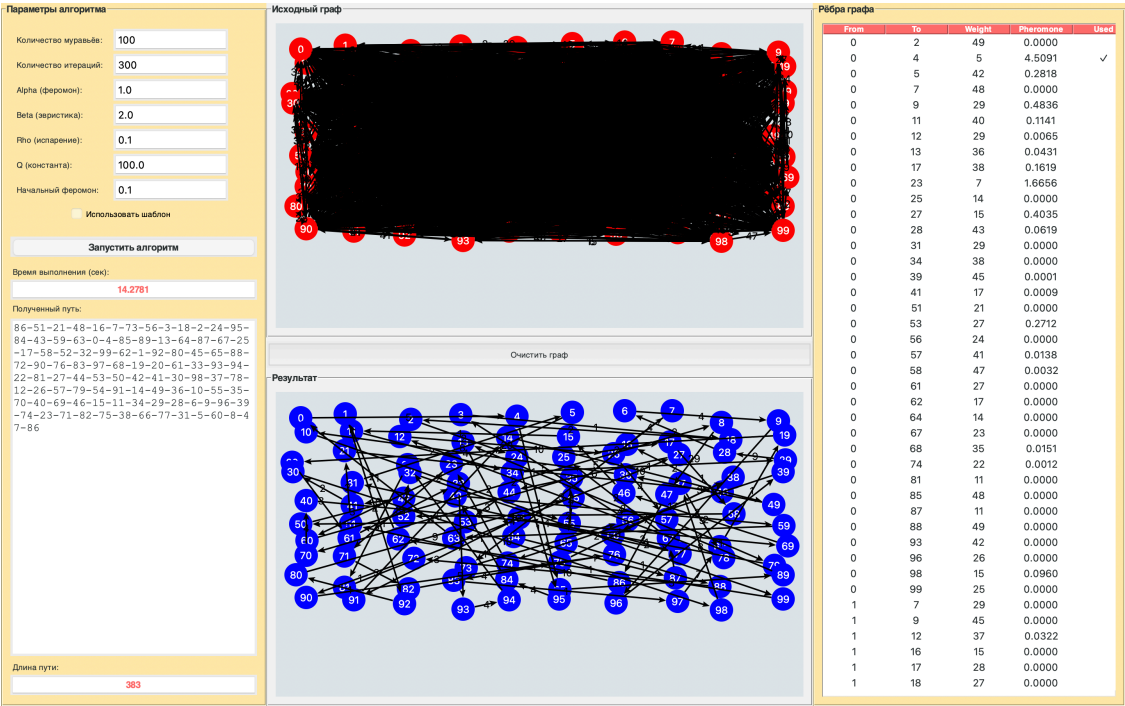


Рис 5. Граф с 100 вершинами (базовый)

Модификация (с шаблоном)

Table 13: Модификация с шаблоном (100 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	410	5.9376
2	438	5.7935
3	420	5.7390
4	404	5.1710
5	409	5.5423

Найденный путь (испытание 1): 0-25-39-74-16-19-68-57-43-17-98-82-36-92-76-24-99-70-58-28-91-54-71-33-38-65-64-31-11-73-69-21-90-8-67-83-56-87-30-15-51-34-4-41-53-94-18-37-52-32-85-89-2-6-62-44-66-49-55-96-59-10-12-47-84-93-61-48-72-3-35-95-80-63-7-78-5-22-40-14-1-29-27-42-45-46-60-77-26-9-79-50-81-97-13-88-75-23-86-20-0

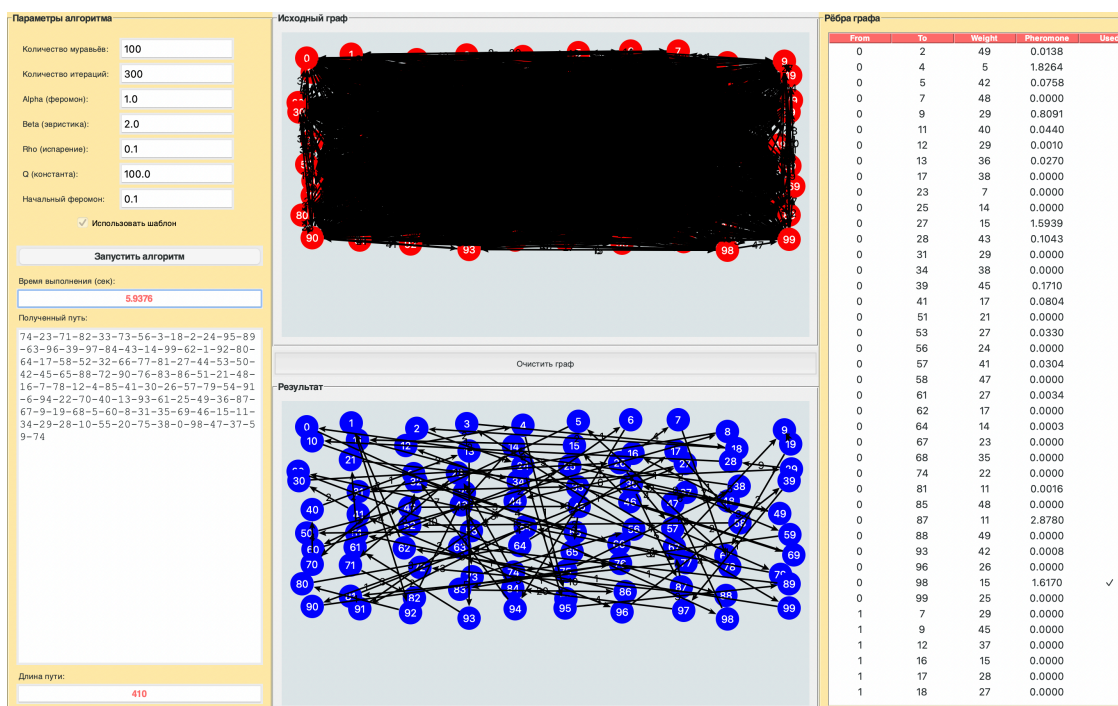


Рис 6. Граф с 100 вершинами (модификация)

5.2.4 Граф с 500 вершинами и 100000 рёбрами

Базовый алгоритм

Table 14: Базовый алгоритм (500 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	906	537.1154
2	1002	547.1233
3	854	527.6523
4	940	517.5422
5	870	507.4321

Найденный путь (испытание 1): 244-409-317-292-471-67-388-405-474-496-432-108-116-69-52-472-291-43-211-389-429-...-83-486-410-413-112-56-481-44-185-335-318-353-194-293-125-299-66-289-222-114-319-160-303-450-51-449-484-23-49-272-275-180-25-294-355-475-47-397-215-418-201-448-359-146-244

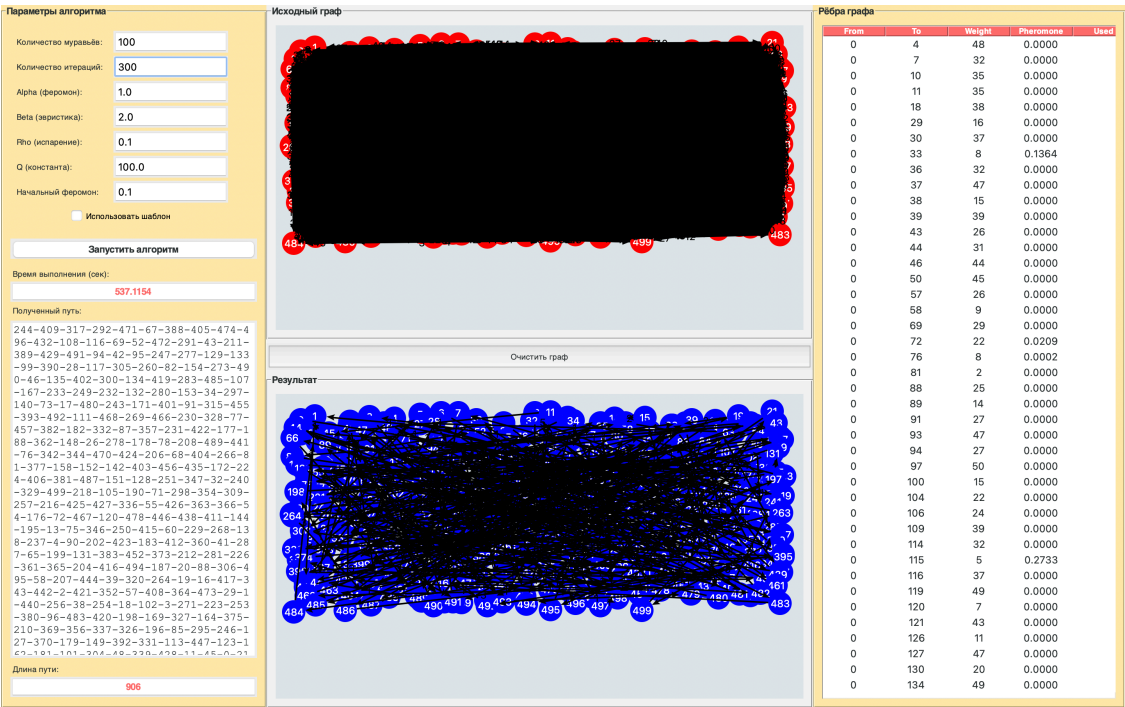


Рис 7. Граф с 500 вершинами (базовый)

Модификация (с шаблоном)

Table 15: Модификация с шаблоном (500 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	862	192.6354
2	828	197.6354
3	833	190.6303
4	851	191.6542
5	874	197.4323

Найденный путь (испытание 1): 468-480-243-171-232-390-406-354-154-273-457-418-97-221-63-27-431-436-462-266-360-...-470-347-486-433-24-89-423-454-415-217-395-282-13-396-139-401-335-420-140-381-373-127-468



Рис 8. Граф с 500 вершинами (модификация)

5.3 Анализ влияния гиперпараметров (граф с 100 вершинами и 3000 рёбрами)

5.3.1 Количество муравьёв

Базовый алгоритм

Table 16: Базовый алгоритм (влияние количества муравьёв)

Количество муравьёв	Длина пути	Время выполнения (с)
2	758	0.0934
10	624	0.4285
50	479	2.1100
250	446	10.4714
1250	408	52.7340

Модификация (с шаблоном)

Table 17: Модификация с шаблоном (влияние количества муравьёв)

Количество муравьёв	Длина пути	Время выполнения (с)
2	699	0.0400
10	586	0.1512
50	504	0.7685
250	441	4.3798
1250	422	22.2341

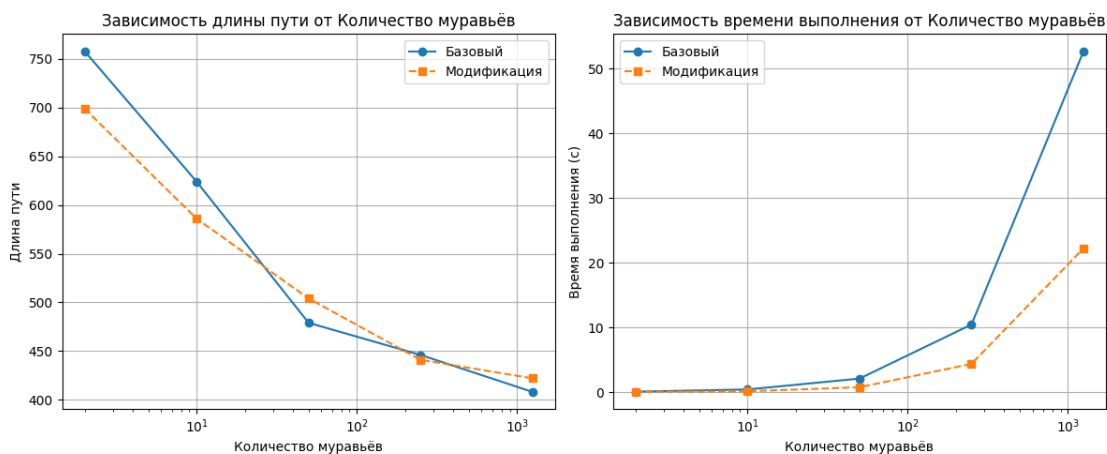


Рис 9. Зависимость длины пути и времени выполнения от количества муравьёв

5.3.2 Количество итераций

Базовый алгоритм

Table 18: Базовый алгоритм (влияние количества итераций)

Количество итераций	Длина пути	Время выполнения (с)
1	921	0.0418
10	525	0.4226
100	470	4.1846
1000	401	42.2660

Модификация (с шаблоном)

Table 19: Модификация с шаблоном (влияние количества итераций)

Количество итераций	Длина пути	Время выполнения (с)
1	793	0.0183
10	666	0.1876
100	504	1.4995
1000	455	15.1944

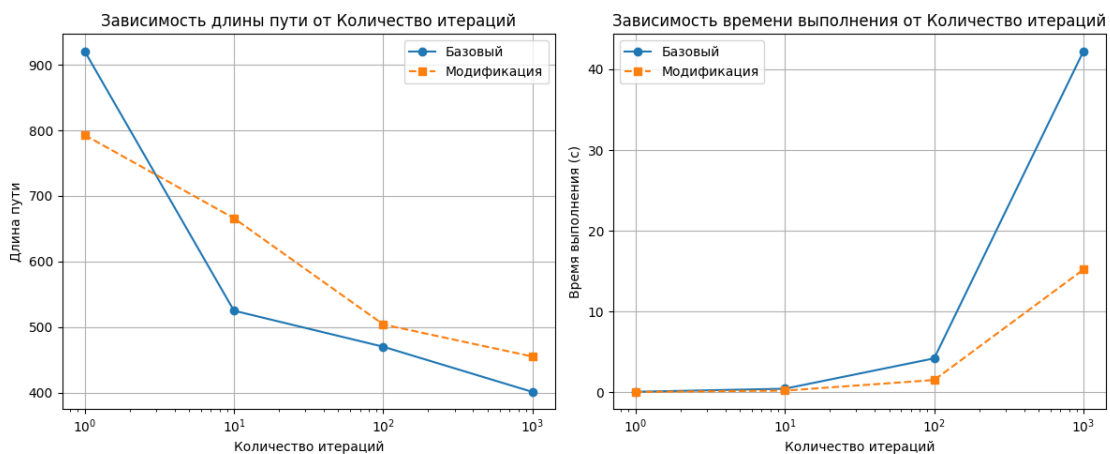


Рис 10. Зависимость длины пути и времени выполнения от количества итераций

5.3.3 Alpha (влияние феромона)

Базовый алгоритм

Table 20: Базовый алгоритм (влияние Alpha)

Alpha	Длина пути	Время выполнения (с)
0.5	511	4.1930
1.0	458	4.1657
1.5	453	4.2770
2.0	434	4.2437

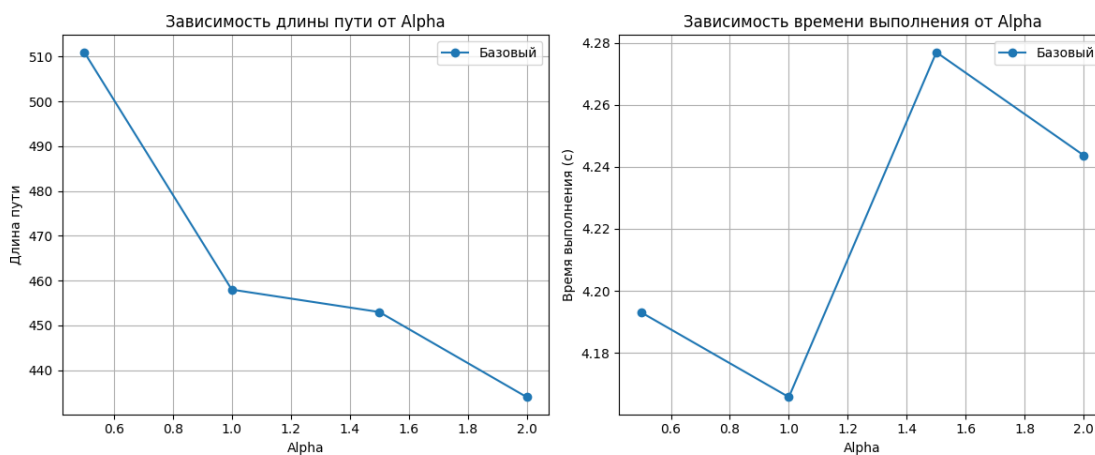


Рис 11. Зависимость длины пути и времени выполнения от Alpha

5.3.4 Beta (влияние эвристики)

Базовый алгоритм

Table 21: Базовый алгоритм (влияние Beta)

Beta	Длина пути	Время выполнения (с)
0.5	710	4.2065
1.0	466	4.2189
1.5	455	4.1907
2.0	448	4.1991

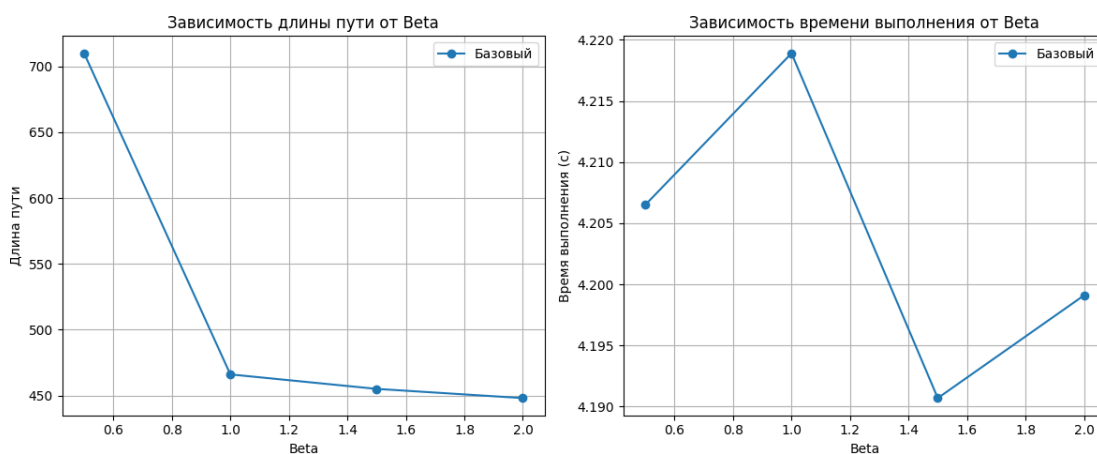


Рис 12. Зависимость длины пути и времени выполнения от Beta

5.3.5 Rho (коэффициент испарения)

Базовый алгоритм

Table 22: Базовый алгоритм (влияние Rho)

Rho	Длина пути	Время выполнения (с)
0.001	483	4.2015
0.01	496	4.1630
0.1	473	4.1603
0.25	415	4.2208
0.5	432	4.2783
0.75	429	4.2648
0.9	432	4.4075
0.99	473	4.4275

Модификация (с шаблоном)

Table 23: Модификация с шаблоном (влияние Rho)

Rho	Длина пути	Время выполнения (с)
0.001	485	1.5010
0.01	503	1.6863
0.1	462	1.1876
0.25	466	1.7159
0.5	523	1.5428
0.75	487	1.5404
0.9	497	1.5640
0.99	572	1.5313

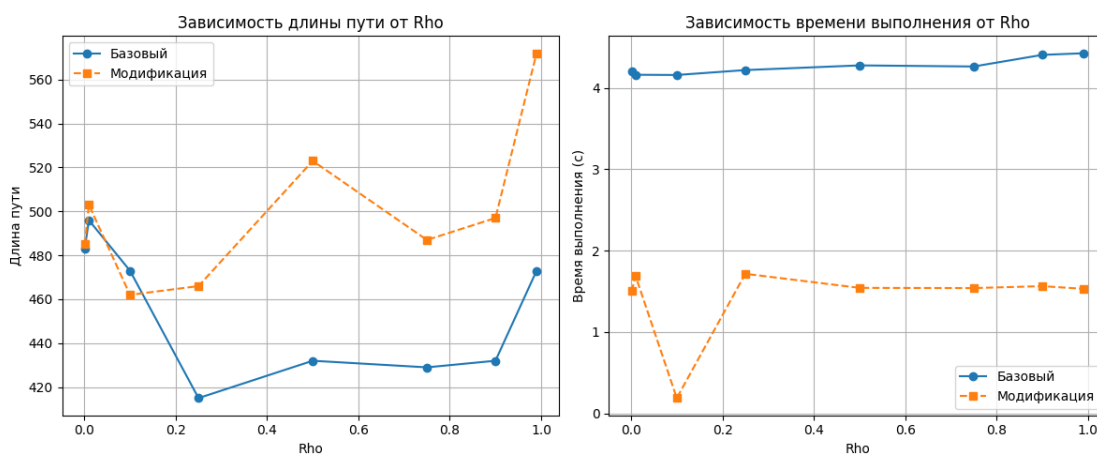


Рис 13. Зависимость длины пути и времени выполнения от Rho

5.4 Анализ результатов

Модификация использует шаблон, основанный на минимальных весах рёбер, снижая сложность выбора следующей вершины с $O(n \cdot k)$ до $O(1)$. Это ускоряет выполнение в 1.4–2.8 раза. На больших графах шаблон также улучшает качество пути (849.6 против 914.4), направляя муравьёв по рёбрам с высокой вероятностью включения в оптимальный маршрут. На малых и средних графах шаблон менее эффективен, так как ограничение выбора вершин снижает полноту перебора.

Базовый муравьиный алгоритм демонстрирует стабильное качество путей на малых и средних графах. Модификация с шаблоном ускоряет выполнение (0.15 с, 0.69 с и 5.64 с соответственно), но на средних графах уступает по длине пути (150.4 и 416.2). На графе с 500 вершинами модификация превосходит базовый алгоритм, находя путь длиной 849.6 за 193.99 с против 914.4 за 527.37 с, что указывает на её эффективность для крупных задач.

Анализ гиперпараметров (см. рис 9-13) показывает, что увеличение числа муравьёв (до 250–1250) и итераций (до 1000) улучшает пути, но экспоненциально увеличивает время. Модификация достигает сопоставимого качества быстрее (например, 422 за 22.23 с против 408 за 52.73 с при 1250 муравьях). Параметры α и β , регулирующие влияние феромонов и эвристики, улучшают пути при значениях 1.5–2.0 без заметного роста времени. Оптимальное $\rho \approx 0.25$ обеспечивает баланс

между сохранением и обновлением феромонов, минимизируя длину пути (415 для базового, 466 для модификации).

Модификация эффективна, когда время выполнения критично, особенно на больших графах, где шаблон компенсирует ограниченный перебор за счёт предварительного выбора перспективных рёбер. Однако на средних графах базовый алгоритм предпочтительнее для задач, где качество пути важнее скорости.

5.5 Сравнение муравьиного алгоритма с алгоритмом имитации отжига и методом ближайшего соседа

5.5.1 Граф с 4 вершинами и 8 рёбрами

Table 24: Сравнение алгоритмов (4 вершины)

Алгоритм	Средняя длина пути	Среднее время выполнения (с)
Базовый ближайшие соседи	Не найден	0.00016
Модификация ближайшие соседи	112	0.00024
Базовый имитация отжига	112	0.00918
Модификация имитация отжига	112	0.10346
Базовый муравьиный	112	0.2124
Модификация муравьиный	112	0.14824

5.5.2 Граф с 20 вершинами и 186 рёбрами

Table 25: Сравнение алгоритмов (20 вершин)

Алгоритм	Средняя длина пути	Среднее время выполнения (с)
Базовый ближайшие соседи	Не найден	0.00014
Модификация ближайшие соседи	155	0.00188
Базовый имитация отжига	382.2	0.10924
Модификация имитация отжига	416.8	0.16988
Базовый муравьиный	137	1.31162
Модификация муравьиный	150.4	0.69086

5.5.3 Граф с 100 вершинами и 3395 рёбрами

Table 26: Сравнение алгоритмов (100 вершин)

Алгоритм	Средняя длина пути	Среднее время выполнения (с)
Базовый ближайшие соседи	Не найден	0.0006
Модификация ближайшие соседи	524	0.03498
Базовый имитация отжига	2450.2	9.48424
Модификация имитация отжига	2501.6	1.3865
Базовый муравьиный	380.8	14.34566
Модификация муравьиный	416.2	5.63668

5.5.4 Граф с 500 вершинами и 100000 рёбрами

Table 27: Сравнение алгоритмов (500 вершин)

Алгоритм	Средняя длина пути	Среднее время выполнения (с)
Базовый ближайшие соседи	Не найден	0.01534
Модификация ближайшие соседи	863	6.67336
Базовый имитация отжига	11740	536.24604
Модификация имитация отжига	12464.6	23.33686
Базовый муравьиный	914.4	527.37306
Модификация муравьиный	849.6	193.99752

Модифицированный метод ближайшего соседа превосходит другие алгоритмы по скорости и обеспечивает конкурентоспособные результаты: на 500 вершинах длина цикла — 863 за 6.67 с ($O(n^3)$). Базовый муравьиный алгоритм лидирует на средних графах (137 на 20 вершинах), а его модификация — на больших (849.6 на 500 вершинах), но время выполнения значительно выше (до 527 с). Имитация отжига точна только на малых графах (112 на 4 вершинах), но на больших резко проигрывает: 11740–12464.6 на 500 вершинах, даже с ускорением в сверхбыстрой версии (23.3 с против 536.2 с).

Базовый метод ближайшего соседа ненадёжен, часто не находя цикл, тогда как его модификация и муравьиный алгоритм всегда дают решение. Модификация муравьиного алгоритма сокращает время и улучшает результат на крупных графах, а сверхбыстрый отжиг ускоряет базовый, но жертвует качеством.

Рекомендуется использовать модифицированный метод ближайшего соседа для оптимального баланса скорости и точности. Муравьиный алгоритм с модификацией предпочтителен на больших графах, если время выполнения не критично. Имитация отжига подходит только для малых графов из-за низкой эффективности на крупных масштабах.

6 Вывод

Базовый муравьиный алгоритм эффективно решает задачу коммивояжера на средних графах, обеспечивая путь длиной 137 за 1.31 с на 20 вершинах, превосходя имитацию отжига (382.2), но уступает модифицированному методу ближайшего соседа (155 за 0.00188 с) по скорости и качеству. Модификация с шаблоном ускоряет выполнение в 1.4–2.8 раза и улучшает результат на больших графах (849.6 за 193.99 с против 914.4 за 527.37 с), но менее точна на средних (150.4). Для большинства задач предпочтителен метод ближайшего соседа, муравьиный алгоритм с модификацией рекомендуется для больших графов при некритичном времени, а имитация отжига подходит лишь для учебных целей.

7 Полезные ссылки

- Репозиторий с лабораторными работами за 4-й семестр

8 Листинг программы

8.1 main.py

```
from colorGUI import LabApp
from graph import Graph
import random
import sys

sys.setrecursionlimit(250000)
RANDOM_SEED = 0

def create_graph(num_nodes, num_edges, canvas_width, canvas_height):
    """
    :param num_nodes:
    :param num_edges:
    :param canvas_width:
    :param canvas_height:
    :return:
    """
    random.seed(RANDOM_SEED)

    graph = Graph(directed=True)

    cols = int(num_nodes**0.5)
    rows = (num_nodes + cols - 1) // cols
    padding = 30

    x_step = (canvas_width - 2*padding) / (cols-1) if cols > 1 else 0
    y_step = (canvas_height - 2*padding) / (rows-1) if rows > 1 else 0

    nodes = []
    for i in range(num_nodes):
        col = i % cols
        row = i // cols
        x = padding + col * x_step + random.randint(-10, 10)
        y = padding + row * y_step + random.randint(-10, 10)
        node = graph.add_node(i)
```

```

        node.x = x
        node.y = y
        nodes.append(node)

    all_possible_edges = [(i, j) for i in range(num_nodes)
                           for j in range(num_nodes) if i != j]
    selected_edges = random.sample(all_possible_edges, num_edges)

    for from_idx, to_idx in selected_edges:
        weight = random.randint(1, 50)
        graph.add_edge(nodes[from_idx], nodes[to_idx], weight)

    return graph

def main():
    # graph = create_graph(num_nodes=4, num_edges=8, canvas_width=676,
    #                       ↪ canvas_height=300)
    # graph = create_graph(num_nodes=20, num_edges=186, canvas_width=676,
    #                       ↪ canvas_height=300)
    # graph = create_graph(num_nodes=100, num_edges=3395, canvas_width=676,
    #                       ↪ canvas_height=300)
    graph = create_graph(num_nodes=500, num_edges=100000, canvas_width=676,
    #                     ↪ canvas_height=300)
    # graph = Graph()

    graph.display()

    app = LabApp(graph)
    app.attributes('-fullscreen', True)
    app.mainloop()

if __name__ == "__main__":
    main()

```

8.2 graph.py

```
class Node:
    """ """

    def __init__(self, value):
        self.value = value
        self.neighbors = []
        self.outgoing_edges = []
        self.x = None
        self.y = None
        self.visited = False

    def add_neighbor(self, node):
        """ """
        self.neighbors.append(node)

    def sorted_edges(self):
        """ """
        return sorted(self.outgoing_edges, key=lambda edge: edge.weight)

    def __repr__(self):
        return f"Node(value={self.value}, (x,y)=({self.x},{self.y}))"

class Edge:
    """ """

    def __init__(self, from_node: Node, to_node: Node, weight: int = 1):
        self.from_node = from_node
        self.to_node = to_node
        self.weight = weight

    def __repr__(self):
        return f"Edge({self.from_node.value} -> {self.to_node.value},  
↔ weight={self.weight})"

class Graph:
    """ """

    def __init__(self, directed=True):
        self.nodes = []
```

```

self.edges = []
self.directed = directed

def add_node(self, value):
    """ """
    node = Node(value)
    self.nodes.append(node)
    return node

def add_edge(self, from_node, to_node, weight=1):
    """ """
    forward_edge = Edge(from_node, to_node, weight)
    self.edges.append(forward_edge)
    from_node.add_neighbor(to_node)
    from_node.outgoing_edges.append(forward_edge)
    if not self.directed:
        reverse_edge = Edge(to_node, from_node, weight)
        self.edges.append(reverse_edge)
        to_node.add_neighbor(from_node)
        to_node.outgoing_edges.append(reverse_edge)

def display(self):
    """ """
    print("Nodes:")
    for node in self.nodes:
        print(node)
    print("Edges:")
    for edge in self.edges:
        print(edge)

```

8.3 `ant_algorithm.py`

```
from graph import Graph, Node
from typing import List, Dict, Tuple, Optional
import random
import time

class Ant:
    """
    ,
    ."""

    def __init__(self, start_node: Node, template: Optional[Dict[Node,
        ↪ Node]] = None):
        """
        .

        :param start_node:
        :param template: ( -> )
        """
        self.current_node = start_node
        self.path = [start_node]
        self.visited = {start_node}
        self.template = template

    def move(self, graph: Graph, alpha: float, beta: float) -> bool:
        """
        ,
        .

        :param graph:
        :param alpha:
        :param beta:
        :return: True, , False
        """
        if len(self.visited) == len(graph.nodes):
            return False

        if self.template and self.current_node in self.template:
            next_node = self.template[self.current_node]
            if next_node not in self.visited:
                self.current_node = next_node
```

```

        self.path.append(next_node)
        self.visited.add(next_node)
        return True

probabilities = []
total = 0
for edge in self.current_node.outgoing_edges:
    if edge.to_node not in self.visited:
        pheromone = getattr(edge, 'pheromone', 0.1)
        heuristic = 1 / edge.weight
        prob = (pheromone ** alpha) * (heuristic ** beta)
        probabilities.append((edge.to_node, prob))
        total += prob

if not probabilities:
    return False

probabilities = [(node, prob / total) for node, prob in
    ↪ probabilities]
r = random.random()
cumulative = 0
for node, prob in probabilities:
    cumulative += prob
    if r <= cumulative:
        self.current_node = node
        self.path.append(node)
        self.visited.add(node)
        return True
return False

def initialize_pheromones(graph: Graph, initial_pheromone: float):
    """
    """
    for edge in graph.edges:
        edge.pheromone = initial_pheromone

def update_pheromones(graph: Graph, ants: List[Ant], rho: float, Q: float):
    """

```

```

        .

:param graph:
:param ants:
:param rho:
:param Q:
"""
#
for edge in graph.edges:
    edge.pheromone *= (1 - rho)

#
for ant in ants:
    if len(ant.path) == len(graph.nodes) + 1 and ant.path[0] ==
        ↪ ant.path[-1]:
        path_length = calculate_path_length(graph, ant.path)
        if path_length > 0 and path_length != float('inf'):
            delta = Q / path_length
            for i in range(len(ant.path) - 1):
                from_node = ant.path[i]
                to_node = ant.path[i + 1]
                edge = next((e for e in from_node.outgoing_edges if
                    ↪ e.to_node == to_node), None)
                if edge:
                    edge.pheromone += delta

def calculate_path_length(graph: Graph, path: List[Node]) -> int:
    """
    .

:param graph:
:param path:
:return:
    ,
"""
    if len(path) != len(graph.nodes) + 1 or path[0] != path[-1]:
        return float('inf')
    total = 0
    for i in range(len(path) - 1):
        from_node = path[i]

```

```

        to_node = path[i + 1]
        edge = next((e for e in from_node.outgoing_edges if e.to_node ==
            ↪ to_node), None)
        if not edge:
            return float('inf')
        total += edge.weight
    return total

def get_template(graph: Graph) -> Dict[Node, Node]:
    """
        .

    :param graph:
    :return:
    """
    template = {}
    for node in graph.nodes:
        if node.outgoing_edges:
            min_weight = min(edge.weight for edge in node.outgoing_edges)
            min_edges = [edge for edge in node.outgoing_edges if edge.weight
                ↪ == min_weight]
            min_edge = random.choice(min_edges)
            template[node] = min_edge.to_node
    return template

def build_result_graph(graph: Graph, path: List[Node]) -> Graph:
    """
        .

    :param graph:
    :param path:
    :return:
    """
    result_graph = Graph(directed=True)
    node_map = {node.value: result_graph.add_node(node.value) for node in
        ↪ graph.nodes}
    for node in result_graph.nodes:
        original_node = next(n for n in graph.nodes if n.value ==
            ↪ node.value)

```



```

node.x = original_node.x
node.y = original_node.y

for i in range(len(path) - 1):
    from_node = path[i]
    to_node = path[i + 1]
    edge = next((e for e in from_node.outgoing_edges if e.to_node ==
        ↪ to_node), None)
    if edge:
        result_graph.add_edge(node_map[from_node.value],
            ↪ node_map[to_node.value], edge.weight)
return result_graph

def ant_algorithm(
    graph: Graph,
    use_template: bool = False,
    num_ants: int = 10,
    num_iterations: int = 100,
    alpha: float = 1.0,
    beta: float = 2.0,
    rho: float = 0.1,
    Q: float = 100.0,
    initial_pheromone: float = 0.1
) -> Tuple[Graph, int, str, str]:
    """
        '''.

:param graph:
:param use_template:
:param num_ants:
:param num_iterations:
:param alpha:
:param beta:
:param rho:
:param Q:
:param initial_pheromone:
:return: ( , , , )
    """
    start_time = time.time()

```

```

if not graph.nodes or not graph.edges:
    return Graph(), 0, " ", ""

template = get_template(graph) if use_template else None

initialize_pheromones(graph, initial_pheromone)
best_path = None
best_length = float('inf')

for iteration in range(num_iterations):
    ants = [Ant(random.choice(graph.nodes), template) for _ in
             ↪ range(num_ants)]
    for ant in ants:
        while len(ant.visited) < len(graph.nodes):
            if not ant.move(graph, alpha, beta):
                break
        if len(ant.visited) == len(graph.nodes) and ant.path[0] in
           ↪ ant.current_node.neighbors:
            ant.path.append(ant.path[0])
        if len(ant.path) == len(graph.nodes) + 1 and ant.path[0] ==
           ↪ ant.path[-1]:
            path_length = calculate_path_length(graph, ant.path)
            if path_length < best_length:
                best_length = path_length
                best_path = ant.path.copy()
    update_pheromones(graph, ants, rho, Q)

if best_path:
    result_graph = build_result_graph(graph, best_path)
    total_weight = calculate_path_length(graph, best_path)
    path_str = "-".join(str(node.value) for node in best_path)
    return result_graph, total_weight, " ", path_str
return Graph(), 0, " ", ""

```

8.4 colorGUI.py

```
import tkinter as tk from tkinter import simpledialog, messagebox, ttk import
math import time import platform from graph import Graph, Edge from
ant_algorithmimportant_algorithm
```

```
class LabApp(tk.Tk): def init(self, graph): super().init() self.title("") self.geometry("1250x700")
self.columnconfigure(0, weight=1, minsize=200) self.columnconfigure(1,
weight=2, minsize=700) self.columnconfigure(2, weight=1, minsize=400)
Увеличен minsize до 400 self.rowconfigure(0, weight=1)
self.graph = graph self.node_clicked = None self.answer_graph =
None self.tree = None self.path_length_entry = None self.path_text =
None self.output_canvas = None self.time_entry = None
self.setup_styles() self.create_widgets() self.draw_initial_graph()
def setup_styles(self) : """ self.BG_COLOR =
"F0F0F0" self.ACCENT_1 = "FF6B6B" self.ACCENT_2 =
"4ECDC4" self.DARK_TEXT = "2D3436" self.LIGHT_TEXT =
"FFFFFF" self.PANEL_COLOR = "FFEEA7" self.BTN_COLOR =
"55EFC4" self.CANVAS_BG = "DFE6E9"
style = ttk.Style() style.theme_use("default") style.configure("Treeview.Hheading"
self.ACCENT_1, foreground = self.LIGHT_TEXT, font =
('Helvetica', 10, 'bold')) style.configure("Treeview", background =
"FFFFFF", fieldbackground = "FFFFFF", foreground =
self.DARK_TEXT) style.map("Treeview", background =
[('selected', self.ACCENT_2)])
def create_widgets(self) : """ left_frame =
tk.LabelFrame(self, padx = 10, pady = 10, text = "", bg =
self.PANEL_COLOR, fg = self.DARK_TEXT, font =
('Helvetica', 12, 'bold')) left_frame.grid(row = 0, column = 0, sticky =
"nsew")
Параметры алгоритма param_frame = tk.Frame(left_frame, bg =
self.PANEL_COLOR) param_frame.pack(fill = "x", pady = 5)
params = [ ("Количество муравьёв:", "num_ants", "100"), (" :
```

```

", "numiterations", "300"), ("Alpha()    :   ", "alpha", "1.0"), ("Beta()    :
", "beta", "2.0"), ("Rho()   :   ", "rho", "0.1"), ("Q()   :   ", "Q", "100.0"), ("   :
", "initialpheromone", "0.1"),]

    for    i,    (labeltext, varname, default)in enumerate(params)                :
tk.Label(paramframe, text = labeltext, bg = self.PANELCOLOR, fg =
self.DARKTEXT, font = ('Helvetica', 10)).grid(row = i, column =
0, sticky = "w", padx = 5, pady = 2)

    entry          =          tk.Entry(paramframe, width          =
15)entry.insert(0, default)entry.grid(row = i, column = 1, padx =
5, pady = 2)setattr(self, f"varnameentry", entry)

    self.usetemplatevar = tk.BooleanVar()tk.Checkbutton(paramframe, text =
"", variable = self.usetemplatevar, bg = self.PANELCOLOR, font =
('Helvetica', 10)).grid(row = len(params), columnspan = 2, pady = 5)

    Кнопка запуска tk.Button( leftframe, text = "", command =
self.startalgorithm, bg          =          self.BTNCOLOR, fg          =
self.DARKTEXT, activebackground = self.ACCENT2, font =
('Helvetica', 12, 'bold')).pack(fill = "x", pady = 10)

    Поле времени выполнения tk.Label( leftframe, text = "" ) :
", bg = self.PANELCOLOR, fg = self.DARKTEXT, font =
('Helvetica', 10)).pack(anchor = "w")

    self.timeentry          =          tk.Entry(leftframe, state          =
"readonly", bg = "FFFFFF", fg = self.ACCENT1, font =
('Helvetica', 12, 'bold'), justify = "center", readonlybackground =
"FFFFFF")self.timeentry.pack(fill          =          "x", pady          =
2)self.timeentry.bind("< Button - 3 >
", self.showcontextmenu)self.timeentry.bind("< Control - c >
", lambda e : self.copytime())if platform.system() == "Darwin" :
self.timeentry.bind("< Command - c > ", lambda e : self.copytime())

    Результаты tk.Label( leftframe, text = " : ", bg =
self.PANELCOLOR, fg = self.DARKTEXT, font =
('Helvetica', 10)).pack(anchor = "w")

    self.pathtext          =          tk.Text(leftframe, height          =          5, state          =
"disabled", bg          =          "FFFFFF", fg          =          "2D3436", font          =

```

```
(CourierNew, 14))self.path_text.pack(fill = "both", pady = 5, expand =
True)
```

```
tk.Label( left_frame, text = " : ", bg = self.PANEL_COLOR, fg =
self.DARK_TEXT, font = (Helvetica, 10)).pack(anchor = "w")
```

```
self.path_length_entry = tk.Entry(left_frame, state =
"readonly", bg = "FFFFFF", fg = self.ACCENT_1, font =
(Helvetica, 12, 'bold'), justify = "center", readonlybackground =
"FFFFFF")self.path_length_entry.pack(fill = "x", pady =
2)self.path_length_entry.bind("< Button - 3 >
", self.show_context_menu)self.path_length_entry.bind("< Control - c >
", lambda e : self.copy_path_length())if platform.system() == "Darwin" :
self.path_length_entry.bind("< Command - c > ", lambda e :
self.copy_path_length())
```

```
Центральная панель с графами center_frame = tk.Frame(self, bg =
self.BG_COLOR)center_frame.grid(row = 0, column = 1, sticky = "nsew")
```

```
Входной граф input_graph_frame = tk.LabelFrame(center_frame, text =
"", padx = 10, pady = 10, bg = self.BG_COLOR, fg =
self.DARK_TEXT, font = (Helvetica, 12, 'bold'))input_graph_frame.pack(fill =
"both", expand = True, side = tk.TOP)
```

```
self.input_canvas = tk.Canvas(input_graph_frame, bg =
self.CANVAS_BG, highlightthickness = 0)self.input_canvas.pack(fill =
"both", expand = True)self.input_canvas.bind("< Button - 1 >
", self.on_canvas_click)
```

```
Кнопка очистки tk.Button( center_frame, text =
"", command = self.clear_graph, bg = self.ACCENT_1, fg =
self.DARK_TEXT, activebackground = "FF8787", font =
(Helvetica, 10), height = 2).pack(fill = "x", pady = 5)
```

```
Выходной граф output_graph_frame = tk.LabelFrame(center_frame, text =
"", padx = 10, pady = 10, bg = self.BG_COLOR, fg =
self.DARK_TEXT, font = (Helvetica, 12, 'bold'))output_graph_frame.pack(fill =
"both", expand = True, side = tk.TOP)
```

```
self.output_canvas = tk.Canvas(output_graph_frame, bg =
self.CANVAS_BG, highlightthickness = 0)self.output_canvas.pack(fill =
```

```
"both", expand = True)
```

```
Правая панель с таблицей ребер right_frame =  
tk.LabelFrame(self, text = "", padx = 10, pady = 10, bg =  
self.PANEL_COLOR, fg = self.DARK_TEXT, font =  
(('Helvetica', 12, 'bold')))right_frame.grid(row = 0, column = 2, sticky =  
"nsew")
```

```
self.tree = ttk.Treeview(right_frame, columns =  
("from", "to", "weight", "pheromone", "used"), pheromoneshow =  
"headings", height = 25)
```

```
for col in ("from", "to", "weight", "pheromone", "used"):  
self.tree.heading(col, text=col.capitalize()) self.tree.column(col, width=80,  
anchor="center")
```

```
scrollbar = ttk.Scrollbar(right_frame, orient = "vertical", command =  
self.tree.yview)self.tree.configure(yscrollcommand = scrollbar.set)
```

```
self.tree.pack(side="left", fill="both", expand=True)  
scrollbar.pack(side="right", fill="y")
```

```
Методы для работы с буфером обмена def copy_to_clipboard(self, text) :  
self.clipboard_clear()self.clipboard_append(text)if platform.system() ==  
"Darwin" : self.update()
```

```
def show_context_menu(self, event) : menu =  
tk.Menu(self, tearoff = 0)menu.add_command(label = "", command =  
self.copy_time)menu.tk.call("tk_popup", menu, event.x_root, event.y_root)
```

```
def copy_time(self) : text = self.time_entry.get()self.copy_to_clipboard(text)
```

```
def copy_path_length(self) : text =  
self.path_length_entry.get()self.copy_to_clipboard(text)
```

```
def on_canvas_click(self, event) : """"""x, y = event.x, event.y node =  
self.find_node_at(x, y)
```

```
if node: if self.node_clicked : self.create_dge(self.node_clicked, node)self.node_click  
None else : self.node_clicked = node else : if not self.node_clicked :  
self.create_node(x, y)
```

```
def find_node_at(self, x, y) : """"""for node in self.graph.nodes :  
if abs(node.x - x) < 15 and abs(node.y - y) < 15 : return node return None
```

```
def create_node(self, x, y) : """"""node =
```

```

self.graph.add_node(len(self.graph.nodes))node.x = xnode.y =
ysel.draw_node(node, self.input_canvas, "red")self.update_edges_table()
def create_edge(self, from_node, to_node) : """if from_node == to_node :
return
weight = simpledialog.askinteger( "Вес ребра", f"Введите вес для
from_node.value→to_node.value : ")
if weight: self.graph.add_edge(from_node, to_node, weight)self.draw_edge(Edge(from
def draw_node(self, node, canvas, color) : """x, y =
node.x, node.ycanvas.createoval(x - 15, y - 15, x + 15, y + 15, fill =
color, outline = "")canvas.create_text(x, y, text = str(node.value), fill =
"white")
def draw_edge(self, edge, canvas) : """from_node =
edge.from_nodeto_node = edge.to_node
dx = to_node.x - from_node.xdy = to_node.y - from_node.yangle =
math.atan2(dy, dx)
offset = 15 x1 = from_node.x + offset * math.cos(angle)y1 = from_node.y +
offset * math.sin(angle)x2 = to_node.x - offset * math.cos(angle)y2 =
to_node.y - offset * math.sin(angle)
canvas.create_line(x1, y1, x2, y2, arrow = tk.LAST, width = 2)mid_x =
(x1 + x2)/2mid_y = (y1 + y2)/2canvas.create_text(mid_x, mid_y, text =
str(edge.weight))
def draw_initial_graph(self) : """for node in self.graph.nodes :
self.draw_node(node, self.input_canvas, "red")
for edge in self.graph.edges: self.draw_edge(edge, self.input_canvas)
def clear_graph(self) : """self.graph = Graph(directed =
True)self.input_canvas.delete("all")self.output_canvas.delete("all")self.update_edges
def start_algorithm(self) : """print("start")try : params =
'use_template' : self.use_template_var.get(), 'num_agents' : int(self.num_agents_entry.get()),
if params['rho'] >= 1 or params['rho'] <= 0: raise ValueError("Rho
(испарение) должно быть между 0 и 1")
except ValueError as e: messagebox.showerror("Ошибка", f"Некорректные
параметры: str(e)") return
try: start_time = time.time()result =

```

```

ant_algorithm(self.graph, use_template = params['use_template'], num_ants =
params['num_ants'], num_iterations = params['num_iterations'], alpha =
params['alpha'], beta = params['beta'], rho = params['rho'], Q =
params['Q'], initial_pheromone = params['initial_pheromone']) end_time =
time.time() execution_time = end_time - start_time

self.time_entry.configure(state = "normal") self.time_entry.delete(0, tk.END) se
"readonly")

self.answer_graph, length, status, path = resultself.update_results(length, status,
except Exception as e: messagebox.showerror("Ошибка", str(e))

def update_results(self, length, status, path) :
"""self.path_text.configure(state = "normal") self.path_text.delete("1.0", tk.END)
"disabled")

self.path_length_entry.configure(state = "normal") self.path_length_entry.delete(0
"readonly")

messagebox.showinfo("Результат", status)

def draw_result_graph(self) : """self.output_canvas.delete("all")
for node in self.answer_graph.nodes : self.draw_node(node, self.output_canvas, "blue"
for edge in self.answer_graph.edges : self.draw_edge(edge, self.output_canvas)
def update_edges_table(self) : """for item in self.tree.get_children() :
self.tree.delete(item)

used_edges = set() if self.answer_graph :
for edge in self.answer_graph.edges : key =
(edge.from_node.value, edge.to_node.value, edge.weight) used_edges.add(key)
sorted_edges = sorted(self.graph.edges, key = lambda edge :
(edge.from_node.value, edge.to_node.value))
for edge in sorted_edges : from_val =
edge.from_node.value to_val = edge.to_node.value weight =
edge.weight pheromone = getattr(edge, 'pheromone', 0.0), 0.0 is_used =
(from_val, to_val, weight) in used_edges

self.tree.insert("", "end", values=(from_val, to_val, weight, f"pheromone : .4f", 4"□"
if name == "main": initial_graph = Graph(directed=True) app = LabApp(initial_graph) app.mainloop()

```